

The Project Graveyard

Unfinished isn’t failed. A place to lay old projects to rest, understand why they stopped, and decide whether they deserve a second life.

Code Variant — Build 01 · Round 1 Idea Submission · **Problem Statement 05**

Rajnish Kumar Verma (solo)

Unfinished projects don’t die, they get forgotten



Context decays

Months later you can’t remember what was done, what was left, or why you stopped.



No honest post-mortem

Projects stall for very different reasons (time, scope, a blocker, lost interest), but nobody records which one.



Only two choices today

“Ignore it” or “delete it”. Restarting, pivoting or salvaging parts is never seriously considered.

Our reading of the problem

Storage isn’t the problem. GitHub already keeps the code.

The missing piece is a decision: *what is this project worth now, and what should I do with it?*

Bury it properly, then decide its fate

1

The Tombstone

Record what the project was, what you wanted to build, what got done and what didn’t.

2

The Autopsy

Tag why it stopped (time, scope creep, tech blocker, lost interest, no users) plus a short note.

3

The Revival Path

A transparent advisor ranks Restart · Continue · Change · Repurpose · Archive, with the reason for each.

Not another project-management app. No tasks, sprints or boards. Just a memory for every idea and a clear, explained decision about what happens next.

Built for people who start more than they finish



Students & hackathon builders

Leave every hackathon with a half-built demo that never gets opened again.

“Was my hackathon idea actually good?”



Working developers

Side projects lose to the day job. They need to know which one deserves weekend time.

“Which project is worth my Saturday?”



Indie hackers & makers

Juggle many ideas at once and need to kill, pivot or relaunch on purpose.

“Should I pivot this or let it go?”

Main features

Project Archive

Every project as a tombstone showing its current status, with search and filters.

Project Profile

What it was, the goal, what's done and unfinished, plus its full history.

The Autopsy

Tagged reason for abandonment plus a short note.

Revival Advisor

Five ranked revival paths, each with its "why".

Project Timeline

Started → milestones → abandoned → revisited.

Haunt Me Later

Set a revisit reminder: "haunt me in 30 days".

GitHub Import

Paste a public repo URL to auto-fill stack, last commit and README.

Salvage Yard

Mark reusable parts: components, learnings, assets.

Current status: Resting · Haunting · Revived · R.I.P.

The Revival Advisor explains its reasoning

What it looks at

- How complete it is (%)
- Cause of “death”
- Time since last touched
- Your interest today (1–5)
- Is the idea still relevant?
- Salvageable parts

Path	Suggested when...
Continue	Mostly done, stopped for lack of time, interest still high
Restart	Idea still relevant, but scope creep or tech debt killed it
Change the idea	Interest is high, but the original idea is no longer relevant
Repurpose	Low interest in the original, but reusable parts exist
Archive (R.I.P.)	Low interest and no relevance. Saves an epitaph of lessons learned

Rule-based and transparent. Every suggestion shows its “why”, and the user always makes the final call.

User flow

1 — Bury
Add a project or import it from GitHub

2 — Autopsy
Tag why it stopped

3 — Assess
Rate interest and relevance today

4 — Get paths
Ranked options with reasons

5 — Decide
Revive with 3 next steps, or archive with an epitaph

6 — Revisit
Timeline updates and reminders

↺ Revisit loops back to Assess: a resting project can be re-evaluated any time, and its whole history stays on the timeline.

UI/UX approach

Playful, not morbid

A light graveyard metaphor (tombstones, “Rest in peace”, “Resurrect”) on a calm dark UI with warm lantern accents.

Logging takes under 2 minutes

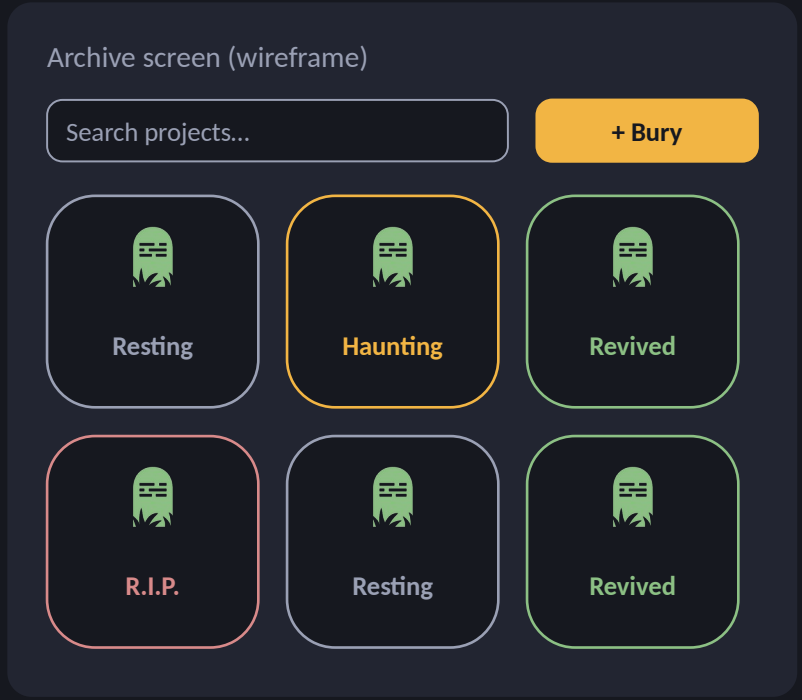
A guided 3-step form with progress and autosave, so filling it in never feels like paperwork.

Status at a glance

Cards over tables, with four states: Resting · Haunting · Revived · R.I.P.

Accessible everywhere

Responsive down to mobile, keyboard-navigable, AA contrast, light/dark mode.



Implementation and technology

React + Vite + TypeScript

Tailwind CSS

Zustand + localStorage

React Router

GitHub REST API (public, no auth)

Framer Motion

Vercel deploy

UI screens

Archive · Profile · Autopsy · Advisor · Timeline

State store

Zustand, persisted to localStorage

Revival engine

Pure TypeScript module with config-driven rules

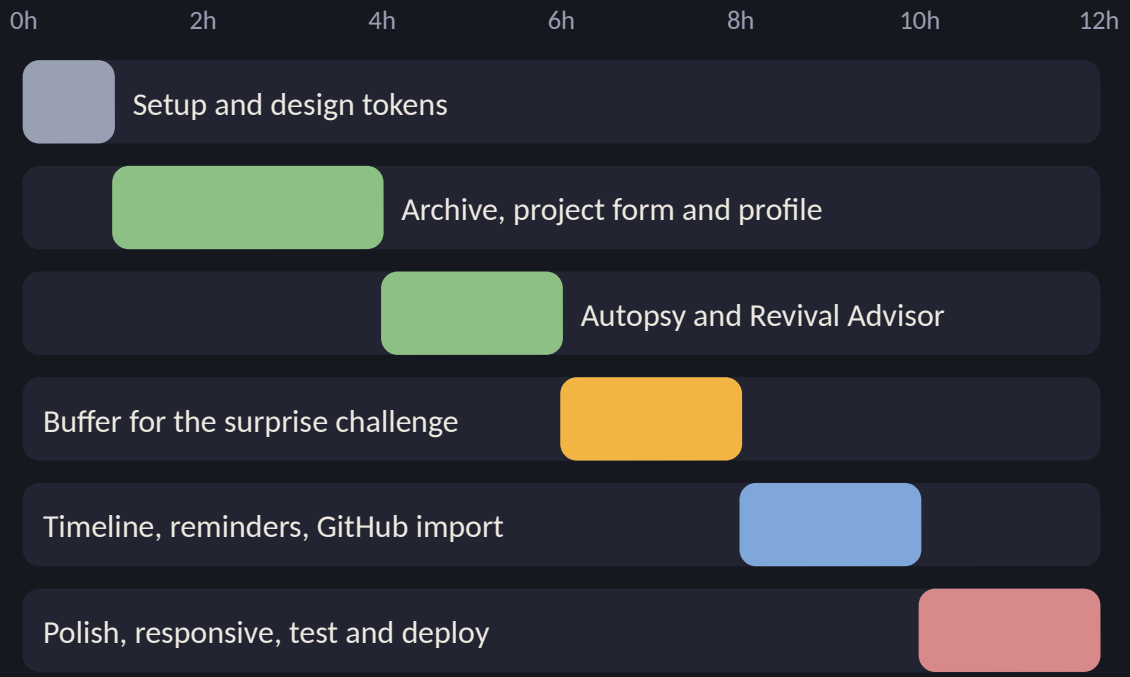
Adapters

Storage · GitHub import



Ready for the Variant Factor: rules, fields and statuses live in config, so the hour-6 twist means editing data, not rewriting screens.

Feasibility: a realistic 12-hour plan



Must-have (MVP)

- Archive and profiles
- Autopsy
- Revival Advisor
- Timeline

Stretch

- GitHub import
- Salvage Yard
- Revisit reminders

No backend needed. Everything runs in the browser.



Every abandoned project deserves a second look

Understand it, decide its fate, and give the good ideas another chance.

Thank you · Rajnish Kumar Verma